

LAPORAN PRAKTIKUM PBO 8



Oleh :

AHMAD FAJRI ARRIZKI

2311532004

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG

1. Buat class baru dengan nama **Customer** di dalam package model. Jika sebelumnya sudah ada class Customer maka silahkan disesuaikan atau diperbarui. Class Customer memiliki beberapa atribut dan sebuah constructor seperti berikut.

```
package model;

public class Pelanggan {
    private String id;
    private String nama;
    private String email;
    private String alamat;
    private String hp;

    // Konstruktor tetap protected atau public
    public Pelanggan(String id, String nama, String email, String alamat, String hp) {
        this.id = id;
        this.nama = nama;
        this.email = email;
        this.alamat = alamat;
        this.hp = hp;
    }
}
```

2. Tambahkan method getter pada class Customer, jika pada project sebelumnya terdapat method setter maka silahkan dihapus karena method setter akan dipindahkan ke Class lain nantinya

```
public String getId() {
    return id;
}

public void setId(String id) {
    this.id = id;
}

public String getName() {
    return nama;
}

public String getEmail() {
    return email;
}

public String getAddress() {
    return alamat;
}

public String getHp() {
    return hp;
}
```

3. Buat sebuah class baru dengan nama **CustomerBuilder** di dalam package model. Class **CustomerBuilder** merupakan builder untuk membuat objek Customer. Dengan menggunakan Builder, kita dapat dengan mudah membuat objek yang memiliki kombinasi atribut yang berbeda

tanpa harus membuat banyak constructor. Selanjutnya, copy semua atribut dari yang ada di class Customer ke dalam class CustomerBuilder dan buat method setter untuk masing-masing atribut tersebut. Di sini kita bisa memberikan default value untuk atribut yang ingin diberikan default value. Kemudian, buat sebuah method dengan nama build dengan return value adalah Customer

```
public class CustomerBuilder {
    private String id;
    private String nama;
    private String email = "";
    private String alamat;
    private String hp;

    // Constructor
    public CustomerBuilder() {
        // Default constructor
    }

    // Setters untuk Builder
    public CustomerBuilder setId(String id) {
        this.id = id;
        return this;
    }

    public CustomerBuilder setNama(String nama) {
        this.nama = nama;
        return this;
    }

    public CustomerBuilder setEmail(String email) {
        this.email = email;
        return this;
    }

    public CustomerBuilder setAlamat(String alamat) {
        this.alamat = alamat;
        return this;
    }

    public CustomerBuilder setHp(String hp) {
        this.hp = hp;
        return this;
    }

    public Pelanggan build() {
        return new Pelanggan(id, nama, email, alamat, hp);
    }
}
```

4. Buat tampilan CRUD Customer menggunakan JFrame di dalam package ui yang diberi nama dengan **CustomerFrame**. Jika sebelumnya **CustomerFrame** sudah ada maka silahkan disesuaikan dan diperbarui jika diperlukan.

The image shows a web form with the following elements:

- Input field for **Nama**
- Input field for **Alamat**
- Input field for **Telepon**
- Input field for **Email**
- A large empty rectangular box below the input fields.
- Buttons for **Simpan** and **Reset** at the bottom.

5. Pada package **dao**, buat interface baru dengan nama **CustomerDao** yang dapat mempermudah dalam melakukan proses CRUD Customer yang terdiri dari menampilkan data customer dari database, menambah, memperbarui, dan menghapus data customer.

```

1 package DAO;
2
3 import java.util.List;
4
5
6 public interface CustomerDAO {
7     public void save(Pelanggan cs);
8     public void update(Pelanggan cs);
9     public void delete(String id);
10    public List<Pelanggan> show();
11 }
12 |

```

6. Buat class baru dengan nama **CustomerRepo** di dalam package **dao** yang akan digunakan untuk mengimplementasikan **CustomerDao** yang telah dibuat sebelumnya. Tambahkan variabel bertipe string sebagai perintah query yang akan digunakan untuk proses CRUD dan juga instansiasi koneksi pada class constructor.

```

public class CustomerRepo implements CustomerDAO {
    private static final Logger logger = Logger.getLogger(CustomerRepo.class);
    private static CustomerRepo instance; // Singleton instance
    private Connection connection;

    private final String insert = "INSERT INTO customer (nama, email, alamat, hp) VALUES (?, ?, ?, ?)";
    private final String select = "SELECT * FROM customer;";
    private final String delete = "DELETE FROM customer WHERE id = ?;";
    private final String update = "UPDATE customer SET nama = ?, email = ?, alamat = ?, hp = ? WHERE id = ?;";

    // Private constructor untuk Singleton
    public CustomerRepo() {
        connection = Database.getInstance().getConnection(); // Mendapatkan koneksi database
    }
}

```

7. Selanjutnya, implementasikan masing-masing method yang terdapat pada interface **CustomerDao** yaitunya method save, update, delete, dan show. Berikut adalah implementasi method **show** pada class **CustomerRepo**

```

public List<Pelanggan> show() {
    List<Pelanggan> ls = new ArrayList<>();
    try (Statement st = connection.createStatement(); ResultSet rs = st.executeQuery(select)) {
        while (rs.next()) {
            // Menggunakan Builder untuk membangun objek Pelanggan
            Pelanggan cs = new Pelanggan.Builder()
                .setId(rs.getString("id"))
                .setNama(rs.getString("nama"))
                .setEmail(rs.getString("email"))
                .setAlamat(rs.getString("alamat"))
                .setHp(rs.getString("hp"))
                .build();
            ls.add(cs);
        }
    } catch (SQLException e) {
        logger.log(Level.SEVERE, "Error while retrieving customer data from the database", e);
    }
    return ls;
}

```

8. Implementasi method **save** pada class **CustomerRepo**

```

public void save(Pelanggan cs) {
    try (PreparedStatement st = connection.prepareStatement(insert)) {
        st.setString(1, cs.getNama());
        st.setString(2, cs.getEmail());
        st.setString(3, cs.getAlamat());
        st.setString(4, cs.getHp());
        st.executeUpdate();
    } catch (SQLException e) {
        logger.log(Level.SEVERE, "Error while saving customer: " + cs, e);
    }
}

```

9. Implementasikan juga method **update** dan **delete** pada class **CustomerRepo**

```
public void update(Pelanggan cs) {
    try (PreparedStatement st = connection.prepareStatement(update)) {
        st.setString(1, cs.getNama());
        st.setString(2, cs.getEmail());
        st.setString(3, cs.getAlamat());
        st.setString(4, cs.getHp());
        st.setString(5, cs.getId()); // Menggunakan ID pelanggan untuk update
        st.executeUpdate();
    } catch (SQLException e) {
        logger.log(Level.SEVERE, "Error while updating customer: " + cs, e); // Log
    }
}

@Override
public void delete(String id) {
    try (PreparedStatement st = connection.prepareStatement(delete)) {
        st.setString(1, id);
        st.executeUpdate();
    } catch (SQLException e) {
        logger.log(Level.SEVERE, "Error while deleting customer with ID: " + id, e);
    }
}
```

10. Selanjutnya, kembali ke tampilan CRUD Customer. Untuk menampilkan data customer dalam bentuk table pada CustomerFrame kita perlu membuat Class baru dengan nama **TableCustomer** pada package **table**. TableCustomer ini akan meng-**extends** class **AbstractTableModel** yang merupakan class bawaan dari Java. Berikut adalah kode program dari class **TableCustomer**.

```

public class TableCustomer extends AbstractTableModel {
    private List<Pelanggan> customers;
    private String[] columnNames = {"ID", "Nama", "Alamat", "Telepon", "Email"};

    public TableCustomer(List<Pelanggan> customers) {
        this.customers = customers;
    }

    @Override
    public int getRowCount() {
        return customers.size();
    }

    @Override
    public int getColumnCount() {
        return columnNames.length;
    }

    @Override
    public Object getValueAt(int rowIndex, int columnIndex) {
        Pelanggan customer = customers.get(rowIndex);
        switch (columnIndex) {
            case 0:
                return customer.getId();
            case 1:
                return customer.getNama();
            case 2:
                return customer.getAlamat();
            case 3:
                return customer.getHp();
            case 4:
                return customer.getEmail();
            default:
                return null;
        }
    }
}

```

11. Di **CustomerFrame**, tambahkan instance **id**, **ls** yang akan menampung data List Customer dan objek **customerRepo**. Selain itu, kita juga perlu membuat method untuk load data customer dan memasukkannya ke TableCustomer, serta method untuk me-reset form input customer.

```

public class CustomerFrame extends JFrame {
    private static final long serialVersionUID = 1L;
    private JPanel contentPane;
    private JTextField txtCustomerName;
    private JTextField txtCustomerAddress;
    private JTextField txtCustomerPhone;
    private JTextField txtCustomerEmail;
    private.JTable tableCustomers;

    private List<Pelanggan> ls;
    private CustomerRepo customerRepo = CustomerRepo.getInstance(); // E

    private String selectedCustomerId = null; // To store selected custo

    public static void main(String[] args) {
        EventQueue.invokeLater(new Runnable() {
            public void run() {
                try {
                    CustomerFrame frame = new CustomerFrame();
                    frame.setVisible(true);
                    frame.loadTable();
                } catch (Exception e) {
                    e.printStackTrace();
                }
            }
        });
    }

    // Load data into the table
    public void loadTable() {
        ls = customerRepo.show();
        TableCustomer tc = new TableCustomer(ls);
        tableCustomers.setModel(tc);
        tableCustomers.getTableHeader().setVisible(true);
    }

    // Reset form fields and selected customer ID
    public void reset() {
        txtCustomerName.setText("");
        txtCustomerAddress.setText("");
        txtCustomerPhone.setText("");
        txtCustomerEmail.setText("");
    }
}

```

12. Tambahkan kode berikut pada constructor CustomerFrame untuk menampilkan data customer ke dalam table.

```

public CustomerFrame() {
    // .....

    JPanel panel = new JPanel();
    panel.setBounds(6, 250, 576, 357);
    contentPane.add(panel);

    JScrollPane scrollPane = new JScrollPane();
    scrollPane.setBounds(10, 10, 529, 307);
    panel.add(scrollPane);

    tableCustomers = new JTable();
    scrollPane.setViewportView(tableCustomers);
    tableCustomers.addMouseListener(new MouseAdapter() {
        @Override
        public void mouseClicked(MouseEvent e) {
            // ...
        }
    });
}

```

13. Selanjutnya, untuk menambahkan customer baru dan menyimpan datanya ke database ketika tombol **save** diklik, silahkan tambahkan kode berikut. Pada proses ini kita ketika tombol **save** diklik, silahkan tambahkan kode berikut. Pada proses ini kita data inputan pengguna sebelum disimpan ke database

```

JButton btnSave = new JButton("Simpan");
btnSave.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        Pelanggan customer = new CustomerBuilder()
            .setNama(txtCustomerName.getText())
            .setAlamat(txtCustomerAddress.getText())
            .setHp(txtCustomerPhone.getText())
            .setEmail(txtCustomerEmail.getText())
            .build();

        customerRepo.save(customer);
        reset();
        loadTable();
    }
});

```